

Project Structure Overview

CampusCare project is divided into two main folders:

1. backend
2. frontend

Backend structure

backend/

The backend folder contains the Node.js and Express.js server. It is responsible for handling authentication, user roles, issue management, worker management, admin management, notifications, image uploads, and database communication.

database/

Contains the database SQL files used to create and prepare the Supabase PostgreSQL database.

- `schema.sql`: Contains the database structure, including tables, relationships, and required database definitions.
- `seed.sql`: Contains initial sample or required data, such as categories, locations, users, or other startup records.

node_modules/

Contains the installed backend dependencies. This folder is generated automatically after running `npm install` and should not be edited manually.

src/

Contains the main backend source code.

config/

Contains configuration files used by the backend.

- `cloudinary.js`: Contains the Cloudinary configuration used for storing uploaded issue photos and completion photos.

controllers/

Contains the main backend logic for each feature. Controllers handle requests, process data, communicate with the database, and return responses to the frontend.

It includes:

- `authController.js`: Handles registration, login, password reset, and authentication-related logic.
- `issueController.js`: Handles issue submission, issue updates, assignment, status changes, comments, notifications, and issue management.
- `userController.js`: Handles user-related logic such as viewing users and activating or deactivating accounts.

db/

Contains the database connection file.

- `index.js`: Connects the backend to the Supabase PostgreSQL database using the database connection string.

middleware/

Contains middleware functions used before requests reach the controllers.

It includes:

- `authMiddleware.js`: Verifies JWT tokens and protects private routes.
- `uploadMiddleware.js`: Handles image uploads before sending them to Cloudinary.

routes/

Contains the API route files. Each route file groups related endpoints.

It includes:

- `authRoutes.js`: Contains authentication routes such as register, login, and password reset.
- `issueRoutes.js`: Contains issue-related routes such as submit issue, view issues, assign worker, update status, add comments, upload completion photo, and notifications.
- `userRoutes.js`: Contains user management routes such as viewing users and activating or deactivating accounts.

.env

Contains private environment variables such as the Supabase database connection string, JWT secret, and Cloudinary credentials. This file should not be uploaded publicly.

index.js

The main backend entry file. It starts the Express server, connects middleware, registers routes, and runs the backend API.

package.json

Contains backend dependencies, scripts, and project metadata.

package-lock.json

Stores the exact installed versions of backend dependencies.

Frontend Structure

frontend/

The frontend folder contains the React Native Expo mobile application. It is responsible for the user interface, navigation, role-based screens, forms, image selection, and communication with the backend API.

.expo/

Contains Expo-generated project files. This folder is created automatically when running the Expo app.

assets/

Contains application images, icons, and other static resources used in the mobile app.

It includes:

- `icon.png`: The application icon used by Expo.

node_modules/

Contains the installed frontend dependencies. This folder is generated automatically after running `npm install` and should not be edited manually.

src/

Contains the main frontend source code.

navigation/

Contains the navigation files used to move between screens. It handles role-based navigation so each user sees the correct dashboard after login.

It includes:

- AdminNavigator.js: Handles navigation for the System Admin screens.
- CMNavigator.js: Handles navigation for Community Member screens.
- FMNavigator.js: Handles navigation for Facility Manager screens.
- WorkerNavigator.js: Handles navigation for Worker screens.

screens/

Contains all application screens for the different user roles.

It includes:

admin/

Contains System Admin screens.

- AdminUserListScreen.js: Allows the admin to view registered users and activate or deactivate user accounts.

manager/

Contains Facility Manager screens.

- FMDashboardScreen.js: Displays submitted issues and allows the manager to search, filter, and manage issues.
- FMIssueDetailScreen.js: Displays full issue details and allows the manager to assign workers, update status, set priority, delete issues, and manage issue progress.

member/

Contains Community Member screens.

- common.js: Contains shared styling or helper elements used by member screens.
- HomeScreen.js: Displays the Community Member home screen.
- IssueDetailScreen.js: Displays full details of a selected submitted issue.
- IssueStatusScreen.js: Displays issue status information.
- MyIssuesScreen.js: Displays issues submitted by the logged-in Community Member.
- SubmitIssueScreen.js: Allows the Community Member to submit an issue with title, description, category, location, and photo.

worker/

Contains Worker screens.

- AssignedIssuesScreen.js: Displays issues assigned to the logged-in worker.
- IssueDetailScreen.js: Displays the assigned issue details and allows the worker to update progress, add comments, upload a completion photo, and resolve the issue.

Authentication screens

These screens are used by multiple roles:

- LoginScreen.js: Allows users to log in.
- RegisterScreen.js: Allows users to create an account.
- ResetPasswordScreen.js: Allows users to reset their password using the simplified password reset process.

services/

Contains files responsible for connecting the frontend to the backend API.

- api.js: Contains the base API URL and request functions used by the frontend to communicate with the backend.

App.js

The main entry point of the React Native Expo application. It connects the navigation structure and starts the app.

app.json

Contains Expo configuration such as app name, icon, splash screen, and platform settings.

index.js

Registers and starts the Expo React Native application.

package.json

Contains frontend dependencies, scripts, and project metadata.

package-lock.json

Stores the exact installed versions of frontend dependencies.